

ArTE-VLA: Autoregressive Target Effects with Implicit Target Grounding for Vision-Language-Action Models

Abstract—Vision-language-action (VLA) models typically generate robot actions directly from visual observations and language instructions, leaving the intended physical evolution of the scene implicit. We introduce ArTE-VLA, a VLA that predicts how the language-referred target should change before generating an action. Given multi-view RGB observations and an instruction, ArTE-VLA infers an implicit target representation and autoregressively predicts target effects at four future horizons. Each effect describes translation, rotation, linear and angular velocity, and activity. The predicted effects are embedded as tokens and appended to the visual-language prefix, allowing the action expert to condition on a compact physical future. Simulator states provide effect supervision during training but are not required at inference. We evaluate one seed-1000 training checkpoint per method under three rollout seeds. On all ten LIBERO-Object tasks, ArTE-VLA improves the success rate of a recipe-matched $\pi_{0.5}$ baseline from 70.9% to 84.3%, a 13.4 percentage-point gain over 1,500 paired episodes per method. On RoboCasa PickPlace-5, it improves aggregate success from 63.3% to 67.0%. Fixed-noise interventions further show that changing the predicted effect values produces consistent changes in generated actions. Qualitative visualizations further compare predicted effects with simulator-derived targets and examine spatial sensitivity. These results suggest that a compact, target-centric future representation can provide an effective and inspectable interface between VLA perception and control.

I. INTRODUCTION

Vision-language-action models have emerged as a promising route toward general robot policies [1], [2], [3], [4]. By combining pretrained visual-language representations with large collections of robot demonstrations, recent VLAs can follow open-vocabulary instructions and transfer across objects, tasks, and environments [5], [6], [1]. Most existing architectures, however, are optimized to map the current observation directly to an action sequence [7], [8]. The physical outcome that connects an instruction to an action—which object should move, in what way, and over what time scale—remains latent inside the policy.

This direct formulation creates two related difficulties. First, the action expert must simultaneously resolve language grounding, infer task-relevant dynamics, and generate precise controls [9], [10]. Second, the resulting computation is difficult to inspect: a successful rollout does not reveal whether the policy anticipated the intended object motion or relied on a less robust correlation [9], [11]. Predictive world models and visual chain-of-thought methods address this problem by forecasting future images or point trajectories before acting [12], [13], [14], [15]. Although expressive, these intermediate representations can be high-dimensional and need not isolate the physical state of the object that is

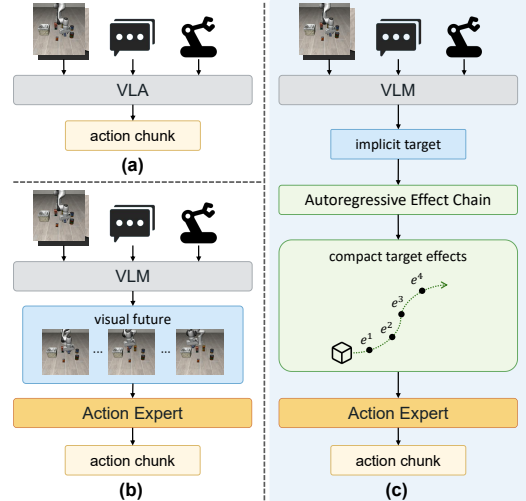


Fig. 1. Comparison of action-generation paradigms. (a) Direct VLA policies map multimodal context to an action chunk without an explicit prediction of scene evolution. (b) A world action model [9] predicts high-dimensional visual futures before action generation. (c) ArTE-VLA instead infers an implicit language-grounded target and predicts a compact chain of structured target effects that conditions the action expert.

central to the instruction [16].

As illustrated in Fig. 1, we propose ArTE-VLA, which inserts a compact target-effect bottleneck into a pretrained VLA model. Rather than predicting a complete future image, the model compresses its visual-language prefix into an implicit representation of the primary target and predicts target effects at four horizons. Each prediction contains a 3-D translation, a 3-D rotation, linear and angular velocity, and an activity logit. The predictions form an autoregressive chain: the effect at one horizon is used to predict the next. Horizon-specific hidden states and effect values are then projected into tokens that condition the flow-matching action expert.

To physically ground this bottleneck, we construct simulator-derived target effects as privileged labels. We recover object states from the simulator states stored in expert demonstrations and construct aligned multi-horizon effect labels. These labels ground the compact bottleneck in intended object motion at multiple horizons, complementing the end-to-end action objective.

Our main contributions are:

- We introduce a compact target-effect bottleneck for VLA policies. The bottleneck predicts a four-horizon autoregressive future and directly conditions the action expert.
- We develop a replay-based labeling pipeline that adds

aligned target effects to robot demonstrations while preserving their original images, actions, states, and language instructions.

- We show that ArTE-VLA outperforms a matched $\pi_{0.5}$ [5] baseline on simulation tasks. Context-preserving interventions test action sensitivity to predicted effect values and their temporal order, while qualitative visualizations inspect effect fidelity and spatial sensitivity.

II. RELATED WORK

A. Vision-Language-Action Models

VLA models transfer pretrained vision-language priors to robot control through diverse manipulation trajectories [17]. RT-1 demonstrated scalable language-conditioned multi-task robot control, while RT-2 transferred Internet-scale vision-language pretraining to action-token generation [18], [19]. Octo supports language or goal-image conditioning and adaptation to new observation and action spaces [20], while OpenVLA emphasizes an open, scalable VLA that can be fine-tuned for downstream manipulation [17]. Flow-matching generative models learn continuous vector fields without discretizing the generated variables [21]. In robotics, π_0 couples a vision-language backbone to a continuous action expert [22], and $\pi_{0.5}$ extends this family with heterogeneous co-training for open-world manipulation [5]. These systems primarily map the current multimodal context directly to actions. ArTE-VLA uses $\pi_{0.5}$ [5] as its backbone but adds a jointly supervised physical prediction interface that is consumed by the otherwise unchanged action expert.

B. Predictive Representations for Control

Predictive representations make intended scene evolution explicit before or alongside action generation [23]. GR-2 jointly learns video and action prediction from web videos and robot trajectories [24]; CoT-VLA generates future visual goals before actions [12]; and WorldVLA places image and action generation in a shared autoregressive formulation [13]. These visual futures retain appearance and multi-object context, but also encode scene content irrelevant to the commanded manipulation [25], [26]. In autonomous driving, ChainFlow-VLA forms autoregressive ego-trajectory modes as causal proposals and refines them through VLM-guided residual diffusion [27]. Unlike these approaches, ArTE-VLA neither synthesizes future observations nor refines full trajectory proposals. It predicts a low-dimensional physical change for one language-conditioned target and inserts it into the action context. Thus, prior approaches preserve rich visual context or optimize complete trajectories, whereas target effects expose compact values whose temporal order and influence on actions can be directly tested.

C. Object and Point Motion as Policy Guidance

Object-centered conditioning separates task-relevant entities from the scene [28], [29]. Earlier language-conditioned manipulation methods separate semantic “what” reasoning from spatial “where” prediction [30]. MOO extracts object

cues from the image and command to support manipulation of unseen categories [31], but describes the current target rather than its desired evolution. Motion representations complement this grounding: Any-point Trajectory Modeling forecasts arbitrary image-point tracks for policy learning [14], and Track2Act converts predicted goal-conditioned tracks into object and end-effector motion plans [15]. ArTE-VLA instead predicts one instruction-grounded target’s multi-horizon 3-D translation, rotation, velocities, and activity, then injects these effects as tokens into an end-to-end VLA action expert. Because its target representation is implicit, it is not a standalone detector or calibrated tracker; the structured effects serve as a compact supervision and action-conditioning bottleneck.

III. METHOD

Figure 2 shows the complete computation. ArTE-VLA preserves the original $\pi_{0.5}$ [5] visual-language prefix and adds a target-effect branch between prefix encoding and action generation. This branch infers a compact target latent, predicts four temporally ordered target effects, and projects each prediction with its horizon-specific hidden state into a token appended to the original prefix. The action expert therefore retains the original multimodal context while receiving a structured prediction of the target’s future. During training, simulator replay supervises the effect predictions through an auxiliary loss; at deployment, every conditioning effect is model-predicted and no simulator-derived quantity is required.

A. Problem Formulation

At time t , the policy receives V RGB observations $I_t = (I_t^{(1)}, \dots, I_t^{(V)})$, proprioceptive state s_t , and a language instruction ℓ . As in $\pi_{0.5}$ [5], the normalized state is discretized and included in the language prompt, while each image is encoded into visual tokens. Let $X_t = \Phi(I_t, s_t, \ell)$ denote the resulting visual-language prefix. A conventional VLA directly models an action chunk $A_t = (a_t, \dots, a_{t+C-1})$ of length C from X_t . We instead introduce an intermediate autoregressive target-effect chain $E_t = (e_t^1, \dots, e_t^H)$ and use the factorization

$$p(A_t, E_t \mid I_t, s_t, \ell) = \left[\prod_{h=1}^H p_\phi(e_t^h \mid e_t^{<h}, X_t) \right] \times p_\theta(A_t \mid X_t, T(E_t)), \quad (1)$$

where $T(E_t)$ embeds the predicted effects into the $\pi_{0.5}$ [5] prefix space. We use $H = 4$ temporal offsets $(k_1, \dots, k_H) = (1, 4, 8, 16)$, corresponding to (0.1, 0.4, 0.8, 1.6) s at the 10-Hz control rate. The intermediate chain asks the model to first represent how the instructed target should evolve and then generate the complete action chunk conditioned on that evolution.

B. Implicit Target Representation

The backbone produces $X_t \in \mathbb{R}^{N \times d}$, including visual patches from both cameras and tokens from the instruction-state prompt. Using a prefix validity mask m_i , we form a

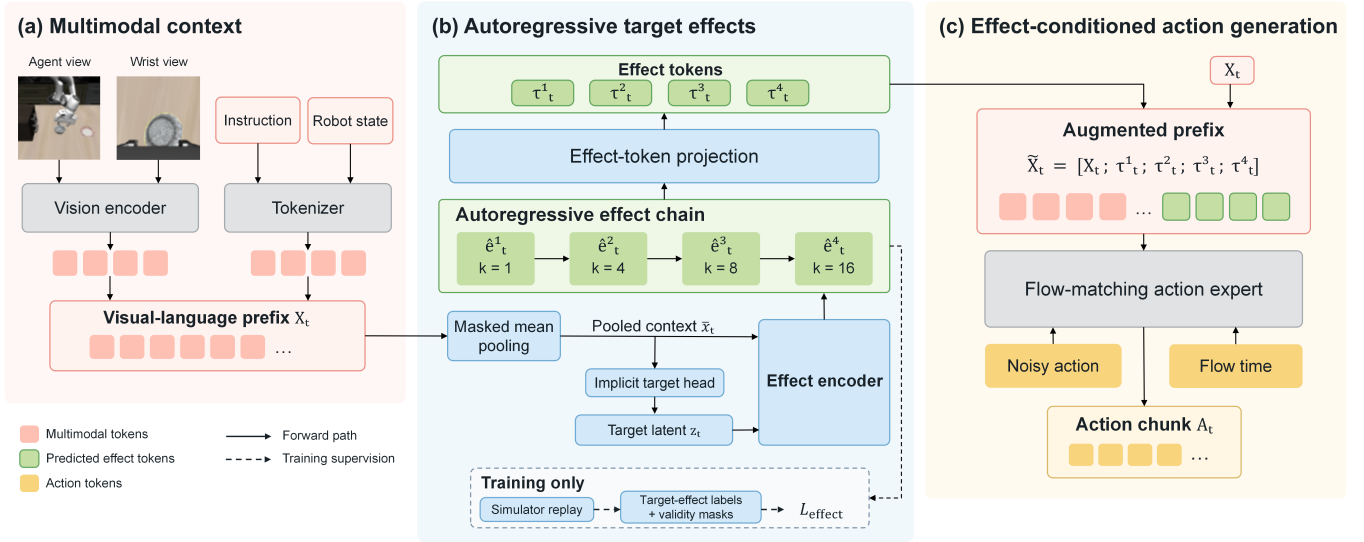


Fig. 2. Overview of ArTE-VLA, illustrated for the dual-view LIBERO [32] setting. (a) Workspace and wrist RGB observations, the instruction, and robot state form the original $\pi_{0.5}$ [5] prefix X_t . (b) Masked pooling and an implicit target head seed a four-horizon autoregressive effect chain; each hidden state and predicted 13-D effect are projected into an effect token. (c) The effect tokens augment X_t and condition the flow-matching action expert, which generates the complete action chunk. The dashed path denotes simulator-derived effect supervision and validity masks.

global context $\bar{x}_t$ and produce one compact target latent:

$$\bar{x}_t = \frac{\sum_{i=1}^N m_i X_{t,i}}{\sum_{i=1}^N m_i}, \quad z_t = f_{\text{target}}(\bar{x}_t) \in \mathbb{R}^{20}. \quad (2)$$

Here f_{target} is a layer-normalized two-layer MLP with a 256-D hidden layer. Our implementation retains the entity-slot dimension of the shared multi-entity predictor, but z_t occupies its sole valid slot and the remaining slots are masked. Joint conditioning on vision, language, and state allows z_t to encode the command-relevant object or articulated part without an explicit detector or object identifier. Importantly, z_t is not regressed to the current simulator state. Its semantics arise only through the downstream future-effect and action objectives, so we call it an *implicit target representation*.

C. Multi-Horizon Effect Chain

The effect chain first combines the target latent with the pooled backbone context,

$$g_t = \text{Enc}_E(f_z(z_t) + f_x(\bar{x}_t)). \quad (3)$$

All effect hidden states are 256-D; Enc_E and Refine_E are two-layer, four-head Transformer encoders. The latter is shared across horizons and applied recursively. With $\hat{e}_t^0 = \mathbf{0}$ and a learned horizon embedding u_h ,

$$c_t^h = \text{Refine}_E(g_t + W_e \hat{e}_t^{h-1} + u_h), \quad (4)$$

$$\hat{e}_t^h = f_E(c_t^h). \quad (5)$$

Here, $c_t^h \in \mathbb{R}^{256}$ is the horizon-specific hidden state used to predict the effect at offset k_h . Thus, a later prediction receives the model's earlier prediction. Each output has the fixed schema

$$\hat{e}_t^h = [\Delta p_t^h, \Delta r_t^h, v_t^h, \omega_t^h, q_t^h] \in \mathbb{R}^{13}, \quad (6)$$

where $\Delta p_t^h \in \mathbb{R}^3$ and $\Delta r_t^h \in \mathbb{R}^3$ describe translation and axis-angle rotation from t to $t + k_h$, $v_t^h, \omega_t^h \in \mathbb{R}^3$ are linear and angular velocities at $t + k_h$, and q_t^h is an activity logit. Translation and velocity are expressed in the end-effector frame at t . With R_t denoting the target orientation, the rotation increment is the logarithmic map of $R_t^\top R_{t+k_h}$. This target-centric effect schema separates intended physical change from the robot controls used to produce it.

D. Conditioning the Action Expert

The 13-D prediction captures explicit physical quantities, while its hidden state retains the horizon-specific context used to produce them. We therefore concatenate each prediction with its hidden state and project the result to the backbone width:

$$\tau_t^h = f_{\text{token}}([c_t^h; \hat{e}_t^h]) \in \mathbb{R}^d. \quad (7)$$

In our $\pi_{0.5}$ [5] backbone, $d = 2048$. The four effect tokens are appended to the original prefix, $\tilde{X}_t = [X_t; \tau_t^1; \dots; \tau_t^4]$, and are visible to the flow-matching action expert. The original prefix is never replaced, so the expert retains direct access to the images, instruction, and state in addition to the predicted future. Moreover, supervision-validity masks are used only in the effect loss: all four predicted tokens remain present during action training and inference. This prevents episode-boundary information from leaking into the policy through a label-derived token mask.

Following $\pi_{0.5}$ [5], let $\epsilon \sim \mathcal{N}(0, I)$ and $\gamma \in (0, 1)$ be the action-flow noise and sampled flow time. Training constructs $A_t^\gamma = (1 - \gamma)A_t + \gamma\epsilon$ with target velocity $u_t = \epsilon - A_t$. The action expert predicts $v_\theta(A_t^\gamma, \gamma | \tilde{X}_t)$ and minimizes

$$\mathcal{L}_{\text{action}} = \mathbb{E}_{A_t, \epsilon, \gamma} \left[\left\| v_\theta(A_t^\gamma, \gamma | \tilde{X}_t) - u_t \right\|_2^2 \right]. \quad (8)$$

At inference, numerical integration starts from Gaussian noise and follows the predicted vector field from $\gamma = 1$ to 0, yielding the complete action chunk.

E. Training Objective

For each valid horizon, simulator replay supplies a continuous target $y_{t,h}^{1:12}$, activity label $y_{t,h}^{\text{act}}$, and validity mask $M_{t,h}$. Let $K_t = \max(1, \sum_h M_{t,h})$. The two effect losses are

$$\mathcal{L}_{\text{reg}} = \mathbb{E}_t \left[\frac{1}{K_t} \sum_h M_{t,h} \frac{1}{12} \sum_{j=1}^{12} \rho(\hat{e}_{t,h}^j - y_{t,h}^j) \right], \quad (9)$$

$$\mathcal{L}_{\text{act}} = \mathbb{E}_t \left[\frac{1}{K_t} \sum_h M_{t,h} \text{BCEWithLogits}(q_t^h, y_{t,h}^{\text{act}}) \right], \quad (10)$$

where ρ is Smooth- L_1 . The full end-to-end objective is

$$\mathcal{L} = \mathcal{L}_{\text{action}} + \lambda_E (\mathcal{L}_{\text{reg}} + \mathcal{L}_{\text{act}}). \quad (11)$$

The main results use $\lambda_E = 1$ for LIBERO-Object [32] and $\lambda_E = 0.1$ for RoboCasa PickPlace-5 [33]. The reported models apply no component- or class-specific reweighting: all 12 continuous dimensions are weighted equally, and positive and negative activity labels receive equal weight before the valid-horizon normalization above. All three loss terms are optimized jointly. Crucially, there is no teacher forcing in the effect recursion, and the action expert is always conditioned on predicted effects rather than replay labels. Training and deployment therefore share the same target-to-effect-to-action path.

F. Replay-Based Effect Supervision

We obtain structured supervision from simulator states during data enrichment; at deployment, the model uses only the same images, instruction, and robot state as the $\pi_{0.5}$ [5] baseline. For every demonstration frame, the enrichment procedure restores the stored flattened MuJoCo state and calls the simulator’s kinematic forward pass; it never steps the recorded action. Let p_t , R_t , v_t , and ω_t denote the target’s position, orientation, linear velocity, and angular velocity in the restored state, and let $R_{\text{ee},t}$ denote the current end-effector orientation. Matching stored states at t and $t + k_h$ yields

$$\Delta p_t^h = R_{\text{ee},t}^\top (p_{t+k_h} - p_t), \quad (12)$$

$$\Delta r_t^h = \text{Log}(R_t^\top R_{t+k_h}), \quad (13)$$

$$v_t^h = R_{\text{ee},t}^\top v_{t+k_h}, \quad (14)$$

$$\omega_t^h = R_{\text{ee},t}^\top \omega_{t+k_h}. \quad (15)$$

The target is marked active if $\|\Delta p_t^h\|_2$ exceeds 1 mm, $\|\Delta r_t^h\|_2$ exceeds 1° , $\|v_t^h\|_2$ exceeds 0.01 m/s, or $\|\omega_t^h\|_2$ exceeds 0.05 rad/s. A horizon is valid only when the same target exists at both frames and $t + k_h$ remains inside the episode.

Task and episode metadata identify the manipulated object’s simulator body. We use the stored demonstration states for LIBERO-Object [32] and the accompanying replay states

for RoboCasa [33], applying the same horizons, thresholds, and validity rules to both. Labels remain aligned with the original demonstration frames. Object identities and task indices select the corresponding supervision labels. Enrichment preserves the original observations, actions, and instructions.

G. Effect Interventions

To test what information reaches the action expert, we intervene after effect prediction while fixing the observation, language, checkpoint, initial action noise, and flow-integration schedule. *Zero-12D* sets only the 12 continuous values in Eq. (6) to zero while preserving each horizon context c_t^h and activity logit. *Reverse* assigns the four 13-D predictions in reverse temporal order, and *Shuffle* substitutes predictions from another sample. For these three effect-value interventions, the altered effects are re-encoded with the receiving sample’s original hidden contexts; its visual, language, and current-state information is never swapped. The resulting tests measure sensitivity to predicted effect values and their temporal organization while preserving the original multimodal context.

IV. EXPERIMENTS

A. Experimental Setup

1) *Benchmark and Data*: We evaluate on LIBERO-Object [32], which contains ten language-conditioned tabletop manipulation tasks. The training set contains 454 available demonstration episodes and 66,984 frames. Replay enrichment yields 253,423 valid target-horizon labels, of which 170,107 are active. We verify frame-level alignment between the original and enriched datasets and observe a maximum action-alignment error of 2.90×10^{-8} . Terminal horizons without a valid successor state are masked.

We also evaluate five atomic tasks from RoboCasa [33], a subset we denote RoboCasa PickPlace-5: Counter to Cabinet, Counter to Stove, Drawer to Counter, Sink to Counter, and Toaster to Counter. Both methods use the same 2,517 human demonstrations (760,518 frames) from `lerobot/robocasa_target_human_unified`, with per-episode instructions from replay metadata. Enrichment yields 2,969,079 valid target-horizon labels, including 1,404,095 active labels, under the rules in Section III-F.

2) *Training Setup*: Both vanilla $\pi_{0.5}$ [5] and ArTE-VLA are initialized from `lerobot/pi05_base` and jointly fine-tune the complete model; we use neither parameter-efficient adaptation nor a frozen vision encoder. Within each benchmark, ArTE-VLA differs from vanilla $\pi_{0.5}$ [5] only by the target head, effect chain, effect-token projection, and the auxiliary loss in Eq. (11). The benchmark-specific settings are summarized in Table I.

All runs use AdamW with peak learning rate 2.5×10^{-5} , weight decay 0.01, $\beta = (0.9, 0.95)$, $\epsilon = 10^{-8}$, and global gradient-norm clipping at 1.0. After the tabulated linear warmup, the learning rate follows cosine decay to 2.5×10^{-6} . All runs use bfloat16 precision, gradient checkpointing, training seed 1000, and no random image augmentation. Square

TABLE I
BENCHMARK-SPECIFIC TRAINING CONFIGURATIONS.

Setting	LIBERO-Object	RoboCasa
		PickPlace-5
Updates	30,000	15,000
GPUs	8	8
Global batch size	64	128
Per-GPU batch size	8	16
Camera views	2	3
Image size (stored/model)	256/224	256/224
Warmup/decay steps	1,000/30,000	500/15,000

images are resized with aspect-preserving padding; LIBERO-Object [32] uses workspace and wrist views, whereas RoboCasa PickPlace-5 [33] uses two agent views and one wrist (eye-in-hand) view.

We evaluate the final checkpoint without validation-based model selection or online environment evaluation during training. Vanilla $\pi_{0.5}$ [5] has 4.143B trainable parameters; ArTE-VLA has 4.149B, adding 5.381M parameters (0.13%). Steady-state checkpoint intervals indicate approximate eight-H20 training times of 12.5 hours for the LIBERO-Object [32] vanilla model and 14.1 hours for ArTE-VLA.

3) *Evaluation Protocol*: For each LIBERO-Object [32] task, we evaluate both policies from the same 50 demonstration-derived initial simulator states under rollout seeds 1000, 2000, and 3000. This produces 500 paired episodes per method and rollout seed, or 1,500 episodes per method overall. Rollouts use synchronous batches of ten environments, ten flow-matching inference steps, and a fixed time horizon. Success is recorded without terminating the episode early. We report per-task and aggregate success, paired rescue and regression counts, and the exact two-sided McNemar test.

For RoboCasa PickPlace-5 [33], each task is evaluated on 20 episodes for each of rollout seeds 1000, 2000, and 3000. This gives 100 episodes per method and rollout seed, or 300 episodes per method overall. For each rollout seed, the two methods use identical environment resets and environment-provided instructions.

4) *Baseline and Ablation*: The primary baseline is recipe-matched vanilla $\pi_{0.5}$ [5]. The only separately trained ablation varies the auxiliary effect-loss weight over $\lambda_E \in \{0.1, 0.3, 1.0\}$ while holding the remaining recipe fixed. We also apply zero-value, reverse-order, and cross-sample shuffle interventions to the same ArTE-VLA checkpoint, together with an identical-condition repeat control. The weight sweep tests optimization sensitivity, while the interventions test whether the trained action expert responds to predicted effect values and their temporal order.

5) *Metrics*: Closed-loop success is the primary metric. We report per-task and aggregate success, together with the absolute percentage-point difference from vanilla $\pi_{0.5}$ [5]. For the paired LIBERO-Object [32] rollouts, we additionally report rescue and regression counts and an exact two-sided McNemar test. Action sensitivity is measured by the fraction of action chunks with any numerical change, the maximum absolute elementwise difference, and the relative root-mean-

square (RMS) distance; the latter two are summarized by their medians across observations. Qualitative diagnostics report projected endpoint error in pixels, activity probability, and the RMS translation change under grid-cell occlusion in millimeters for one selected example. Trainable-parameter overhead is reported in the training setup.

B. Closed-Loop Performance

Table II compares the benchmark-wise best of the three evaluated ArTE-VLA effect-loss weights with recipe-matched vanilla $\pi_{0.5}$ [5] on LIBERO-Object [32] and RoboCasa PickPlace-5 [33]. Using one seed-1000 training checkpoint per method and averaging over three rollout seeds, target-effect conditioning improves mean LIBERO-Object [32] success from 70.9% to 84.3%, an absolute gain of 13.4 percentage points. Pooling the 1,500 paired rollouts, ArTE-VLA produces 380 rescues and 179 regressions; the corresponding exact two-sided McNemar test yields $p = 1.20 \times 10^{-17}$. On RoboCasa PickPlace-5 [33], aggregate success increases from 63.3% to 67.0%, a 3.7-point gain averaged over the same three rollout seeds.

The task-level results reveal two distinct improvement profiles. On LIBERO-Object [32], the aggregate gain is driven by large rescues on Butter and Chocolate Pudding: vanilla $\pi_{0.5}$ [5] achieves only 4.7% and 14.7%, whereas ArTE-VLA reaches 90.0% and 86.7%, respectively. Performance varies across the remaining tasks, including differences of -33.3 and -27.3 percentage points relative to the baseline on Ketchup and Milk, respectively. On RoboCasa PickPlace-5 [33], the gains are distributed across four of the five tasks, led by a 10.0-point improvement on Counter to Cabinet, with Sink to Counter differing by -1.7 points. Together, these results show that target-effect conditioning can provide both large task-specific rescues and broader incremental gains, while its benefit remains task dependent.

C. Effect-Loss Ablation and Action Sensitivity

a) *Effect-loss weight*: Table III reports sensitivity to λ_E across the three rollout seeds; the training recipe is otherwise matched within each benchmark.

All three weights improve over vanilla $\pi_{0.5}$ [5] on LIBERO-Object [32], with the largest gain at $\lambda_E = 1.0$. RoboCasa PickPlace-5 [33] exhibits the opposite ordering: only $\lambda_E = 0.1$ improves over the baseline, whereas 0.3 and 1.0 reduce aggregate success by 1.3 and 5.6 points. This cross-benchmark reversal shows that stronger effect supervision is not uniformly preferable.

b) *Action sensitivity*: As shown in Table IV, we perform a fixed-noise action audit on 32 observations drawn equally from Butter and Chocolate Pudding. Repeating the unmodified forward computation gives zero numerical action difference. Each value intervention changes the action chunk for all 32 samples. Zeroing the 12 continuous effect values yields a median maximum absolute action difference of 0.00956 and a median relative RMS distance of 0.943%. Reversing the horizon order increases these medians to 0.01198 and 1.109%, while cross-sample shuffling yields

TABLE II

SUCCESS (%) ON LIBERO-OBJECT [32] AND ROBOCASA PICKPLACE-5 [33] ACROSS THREE ROLLOUT SEEDS. ENTRIES ARE MEANS ACROSS ROLLOUT SEEDS. FOR EACH ROLLOUT SEED, LIBERO-OBJECT [32] INCLUDES 50 PAIRED EPISODES PER TASK AND ROBOCASA PICKPLACE-5 [33] INCLUDES 20. THE REPORTED ARTE-VLA SETTING USES $\lambda_E = 1$ ON LIBERO-OBJECT [32] AND $\lambda_E = 0.1$ ON ROBOCASA PICKPLACE-5 [33].

Benchmark	Task	$\pi_{0.5}$	ARTE-VLA	Δ vs. $\pi_{0.5}$
LIBERO-Object	Alphabet Soup	77.3	74.7	-2.7
	Cream Cheese	74.0	96.0	+22.0
	Salad Dressing	86.7	87.3	+0.7
	BBQ Sauce	93.3	88.0	-5.3
	Ketchup	97.3	64.0	-33.3
	Tomato Sauce	70.7	90.0	+19.3
	Butter	4.7	90.0	+85.3
	Milk	95.3	68.0	-27.3
	Chocolate Pudding	14.7	86.7	+72.0
	Orange Juice	95.3	98.7	+3.3
	Overall	70.9	84.3	+13.4
RoboCasa PickPlace-5	Counter to Cabinet	41.7	51.7	+10.0
	Counter to Stove	76.7	81.7	+5.0
	Drawer to Counter	40.0	41.7	+1.7
	Sink to Counter	70.0	68.3	-1.7
	Toaster to Counter	88.3	91.7	+3.4
	Overall	63.3	67.0	+3.7

TABLE III

EFFECT-LOSS-WEIGHT ABLATION AVERAGED ACROSS ROLLOUT SEEDS 1000, 2000, AND 3000. ALL LIBERO [32] EVALUATIONS USE A BATCH SIZE OF 10.

λ_E	LIBERO-Object		RoboCasa PickPlace-5	
	Success	Δ	Success	Δ
0.1	78.4	+7.5	67.0	+3.7
0.3	74.8	+3.9	62.0	-1.3
1.0	84.3	+13.4	57.7	-5.6

TABLE IV

FIXED-NOISE EFFECT INTERVENTIONS ON 32 OBSERVATIONS. ACTION METRICS ARE MEASURED RELATIVE TO THE UNMODIFIED COMPUTATION.

Intervention	Intervention content	Max abs. median	Rel. RMS median (%)	Changed (%)
Repeat	Identical-condition repeat	0	0	0
Zero-12D	Zero 12-D future-state values	0.00956	0.943	100
Reverse	Reverse values across horizons	0.01198	1.109	100
Shuffle	Cross-sample substitution	0.01277	1.306	100

0.01277 and 1.306%. These results establish computational dependence of the action expert on the predicted physical values.

D. Qualitative Effect Inspection

Figures 3 and 4 inspect one activity-positive state from the Butter training demonstrations. The example is selected deterministically as the midpoint of the activity-positive frames for this task. For this selected example, the predicted effects closely match the teacher at intermediate horizons: in the workspace-view projection, their target endpoints differ

by only 0.57 and 0.23 pixels at 0.4 and 0.8 s, respectively. At 1.6 s, the prediction preserves the displacement direction while falling 4.71 pixels short of the teacher endpoint. The channel-wise signature likewise captures the dominant increase in translation across horizons, with some underestimation of long-horizon displacement and rotation. All four activity probabilities exceed 0.99998 and agree with the positive labels. Thus, this example exhibits accurate intermediate-horizon effect prediction together with the correct long-horizon motion direction and activity.

Figure 5 complements the value-space inspection with an input-space perturbation test. Following occlusion-based sensitivity analysis [34], we independently replace each cell of an 8×8 grid in the workspace or wrist view with that view’s mean RGB value and measure the RMS change in the predicted translation chain over all four horizons. The strongest workspace-view response lies near the manipulated object, while the wrist-view response is concentrated in the near-field interaction region. Thus, localized image regions influence the implicit effect prediction in both views.

E. Discussion and Limitations

Across both benchmarks, target-effect conditioning improves aggregate success, combining large rescues on Butter and Chocolate Pudding with gains on four of five RoboCasa tasks. The qualitative inspection provides complementary evidence of effect-prediction accuracy: predictions closely match teacher endpoints at intermediate horizons, preserve the long-horizon displacement direction, and correctly identify activity at all four horizons. The spatial analysis further shows that effect predictions are most sensitive to regions near the manipulated object and the near-field interaction.

The benefit nevertheless remains task dependent. The lower results on Ketchup, Milk, and BBQ Sauce could reflect



Fig. 3. Workspace-view projection of four-horizon implicit target effects for one Butter training state. The first row shows simulator teacher effects and the second row shows model predictions. Colored arrows represent target displacement, dashed arrows represent future linear velocity, and short dark segments indicate the projected target x -axis. The current target pose and camera calibration are used only for visualization projection and are not model inputs.

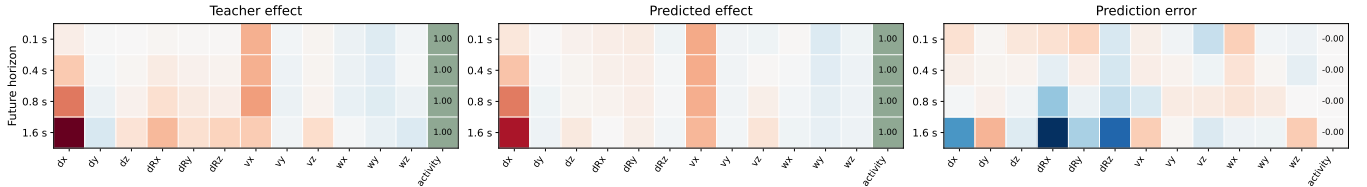


Fig. 4. Implicit target-effect signatures for the state in Fig. 3. Rows denote the four future horizons, and columns encode translation, rotation, linear velocity, angular velocity, and activity. Continuous teacher and prediction channels use shared group-wise scales; the activity column shows the binary teacher label or predicted probability. Future-effect labels are used only to form the teacher and error panels and are not provided to the prediction path.

inaccurate target effects, entanglement of object and task context in the implicit target representation, or competition between effect and visual-language tokens inside the action expert. Distinguishing these explanations requires compar-

ing effect error and action sensitivity on matched rescue and regression episodes. Moreover, the loss-weight sweep reverses the LIBERO [32] ordering on RoboCasa PickPlace-5 [33], indicating that the balance between the action and auxiliary objectives is benchmark dependent even when the target-effect representation is unchanged.

The experimental evidence has several further limits. Table II varies only rollout seeds and uses benchmark-specific reset protocols. The current experiments also lack value-free and structural controls. Consequently, they do not isolate the contribution of explicit physical values from added capacity, autoregressive prediction, or the learned horizon context carried by each effect token. Finally, effect supervision relies on privileged object states available in simulation or instrumented datasets. Extending the method to unconstrained real-world data would require reliable state estimation or pseudo-labeling.

V. CONCLUSION

We presented ArTE-VLA, which augments a VLA model with an implicit target representation and an autoregressive branch that predicts structured target effects at four future horizons. The resulting effect tokens expose translation,

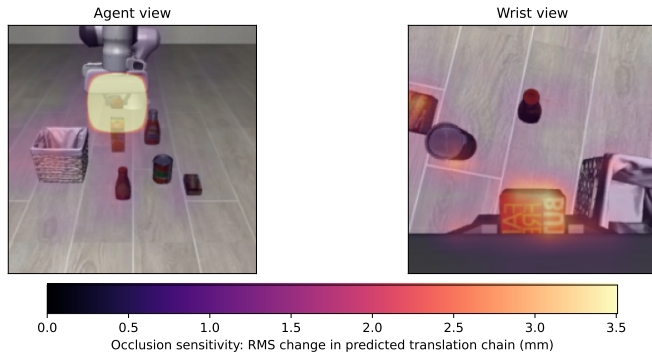


Fig. 5. Spatial occlusion sensitivity of the implicit target-effect prediction for the state in Figs. 3 and 4. Each heatmap reports the RMS change, in millimeters, of the predicted four-horizon target translation chain after independently occluding an 8×8 grid cell. Both views share the same color scale; warmer regions have greater influence on the prediction. No teacher effect, target position, segmentation, or simulator state is used to compute the sensitivity.

rotation, velocity, and activity predictions to the action expert while preserving its original visual-language context. Simulator states supervise the effect branch during training but are not required at inference.

Across three rollout seeds, the reported ArTE-VLA configurations achieve aggregate success rates of 84.3% versus 70.9% for the baseline on LIBERO-Object [32], and 67.0% versus 63.3% on RoboCasa PickPlace-5 [33]. The task-level results include large gains on Butter and Chocolate Pudding and improvements on four of five RoboCasa tasks, although the benefit remains task dependent. Within the fixed-noise audit, effect-value interventions alter all evaluated action chunks, while qualitative visualizations inspect multi-horizon predictions and their spatial sensitivity.

Overall, the results support target-effect prediction as a compact and inspectable interface between VLA perception and control. Future work should evaluate matched structural controls and closed-loop interventions over the full task distribution, and extend effect supervision beyond simulation through reliable state estimation or pseudo-labeling.

REFERENCES

- [1] K. Kawaharazuka, J. Oh, J. Yamada, I. Posner, and Y. Zhu, “Vision-language-action models for robotics: A review towards real-world applications,” *IEEE Access*, 2025.
- [2] X. Li, P. Li, M. Liu, D. Wang, J. Liu, B. Kang, X. Ma, T. Kong, H. Zhang, and H. Liu, “Towards generalist robot policies: What matters in building vision-language-action models,” *arXiv preprint arXiv:2412.14058*, vol. 3, no. 7, p. 8, 2024.
- [3] Y. Ma, Z. Song, Y. Zhuang, J. Hao, and I. King, “A survey on vision-language-action models for embodied AI,” *IEEE Transactions on Neural Networks and Learning Systems*, 2026.
- [4] R. Shao, W. Li, L. Zhang, R. Zhang, Z. Liu, R. Chen, and L. Nie, “Large VLM-based vision-language-action models for robotic manipulation: A survey,” *arXiv preprint arXiv:2508.13073*, 2025.
- [5] P. Intelligence, K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai *et al.*, “ $\pi_0.5$: A vision-language-action model with open-world generalization,” *arXiv preprint arXiv:2504.16054*, 2025.
- [6] M. U. Din, W. Akram, L. S. Saoud, J. Rosell, and I. Hussain, “Vision language action models in robotic manipulation: A systematic review,” *arXiv preprint arXiv:2507.10672*, 2025.
- [7] R. Sapkota, Y. Cao, K. I. Roumeliotis, and M. Karkee, “Vision-language-action (VLA) models: Concepts, progress, applications and challenges,” *arXiv preprint arXiv:2505.04769*, 2025.
- [8] Y. Zhong, F. Bai, S. Cai, X. Huang, Z. Chen, X. Zhang, Y. Wang, S. Guo, T. Guan, K. N. Lui *et al.*, “A survey on vision-language-action models: An action tokenization perspective,” *arXiv preprint arXiv:2507.01925*, 2025.
- [9] S. Wang, J. Shi, Z. Fu, X. He, F. Liu, C. Yang, Y. Zhou, Z. Fei, J. Gong, J. Fu *et al.*, “World action models: The next frontier in embodied AI,” *arXiv preprint arXiv:2605.12090*, 2026.
- [10] W. Guo, Y. Wang, R. Zhou, Y. Zhang, R. Fang, Y. Pang, W. Xu, and G. Huang, “Action with visual primitives,” *arXiv preprint arXiv:2605.22183*, 2026.
- [11] H. Xu, S. Zheng, H. Luo, W. Zhang, Z. Xi, and Z. Lu, “Unmasking the illusion of embodied reasoning in vision-language-action models,” *arXiv preprint arXiv:2604.18000*, 2026.
- [12] Q. Zhao, Y. Lu, M. J. Kim, Z. Fu, Z. Zhang, Y. Wu, Z. Li, Q. Ma, S. Han, C. Finn *et al.*, “CoT-VLA: Visual chain-of-thought reasoning for vision-language-action models,” in *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2025, pp. 1702–1713.
- [13] J. Cen, C. Yu, H. Yuan, Y. Jiang, S. Huang, J. Guo, X. Li, Y. Song, H. Luo, F. Wang *et al.*, “WorldVLA: Towards autoregressive action world model,” *arXiv preprint arXiv:2506.21539*, 2025.
- [14] C. Wen, X. Lin, J. So, K. Chen, Q. Dou, Y. Gao, and P. Abbeel, “Any-point trajectory modeling for policy learning,” *arXiv preprint arXiv:2401.00025*, 2023.
- [15] H. Bharadhwaj, R. Mottaghi, A. Gupta, and S. Tulsiani, “Track2Act: Predicting point tracks from internet videos enables generalizable robot manipulation,” in *European Conference on Computer Vision*. Springer, 2024, pp. 306–324.
- [16] L. Zhong, Y. Liu, Y. Wei, Z. Xiong, S. Liu, and G. Ren, “ACoT-VLA: Action chain-of-thought for vision-language-action models,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2026, pp. 8152–8162.
- [17] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi *et al.*, “Open-VLA: An open-source vision-language-action model,” *arXiv preprint arXiv:2406.09246*, 2024.
- [18] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, J. Dabis, C. Finn, K. Gopalakrishnan, K. Hausman, A. Herzog, J. Hsu *et al.*, “RT-1: Robotics transformer for real-world control at scale,” *arXiv preprint arXiv:2212.06817*, 2022.
- [19] B. Zitkovich, T. Yu, S. Xu, P. Xu, T. Xiao, F. Xia, J. Wu, P. Wohlhart, S. Welker, A. Wahid *et al.*, “RT-2: Vision-language-action models transfer web knowledge to robotic control,” in *Proceedings of the 7th Conference on Robot Learning*. PMLR, 2023, pp. 2165–2183.
- [20] O. M. Team, D. Ghosh, H. Walke, K. Pertsch, K. Black, O. Mees, S. Dasari, J. Hejna, T. Kreiman, C. Xu *et al.*, “Octo: An open-source generalist robot policy,” *arXiv preprint arXiv:2405.12213*, 2024.
- [21] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, “Flow matching for generative modeling,” in *International Conference on Learning Representations*, 2023.
- [22] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter *et al.*, “ π_0 : A vision-language-action flow model for general robot control,” *arXiv preprint arXiv:2410.24164*, 2024.
- [23] J. Ding, Y. Zhang, Y. Shang, Y. Zhang, Z. Zong, J. Feng, Y. Yuan, H. Su, N. Li, N. Sukiennik *et al.*, “Understanding world or predicting future? a comprehensive survey of world models,” *ACM Computing Surveys*, vol. 58, no. 3, pp. 1–38, 2025.
- [24] C.-L. Cheang, G. Chen, Y. Jing, T. Kong, H. Li, Y. Li, Y. Liu, H. Wu, J. Xu, Y. Yang *et al.*, “GR-2: A generative video-language-action model with web-scale knowledge for robot manipulation,” *arXiv preprint arXiv:2410.06158*, 2024.
- [25] X. Xu, H. Li, J. Ye, Y. Chen, J. Zeng, X. Chen, L. Xu, D. Lin, W. Li, and J. Pang, “FutureVLA: Joint visuomotor prediction for vision-language-action model,” *arXiv preprint arXiv:2603.10712*, 2026.
- [26] C. Finn and S. Levine, “Deep visual foresight for planning robot motion,” in *2017 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2017, pp. 2786–2793.
- [27] X. Wang, X. Wang, T. Zhou, G. Chen, X. Gui, Z. Xu, X. Wu, F. Tan, H. Zhou, and M. Yang, “ChainFlow-VLA: Causal flow planning with vision-language models,” *arXiv preprint arXiv:2605.23270*, 2026.
- [28] P. R. Kasu and D. Um, “Object-centric diffusion policies for real-world robotic-arm imitation learning,” *Frontiers in Robotics and AI*, vol. 13, p. 1829436, 2026.
- [29] K. Wang, K. Fan, Y. Fu, S. Lin, H. Luo, D. Seita, Y. Fu, Y.-G. Jiang, and X. Xue, “OCRA: Object-centric learning with 3D and tactile priors for human-to-robot action transfer,” *arXiv preprint arXiv:2603.14401*, 2026.
- [30] M. Shridhar, L. Manuelli, and D. Fox, “CLIPort: What and where pathways for robotic manipulation,” in *Conference on Robot Learning*. PMLR, 2022, pp. 894–906.
- [31] A. Stone, T. Xiao, Y. Lu, K. Gopalakrishnan, K.-H. Lee, Q. Vuong, P. Wohlhart, S. Kirmani, B. Zitkovich, F. Xia *et al.*, “Open-world object manipulation using pre-trained vision-language models,” *arXiv preprint arXiv:2303.00905*, 2023.
- [32] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone, “LIBERO: Benchmarking knowledge transfer for lifelong robot learning,” *Advances in Neural Information Processing Systems*, vol. 36, pp. 44 776–44 791, 2023.
- [33] S. Nasiriany, A. Maddukuri, L. Zhang, A. Parikh, A. Lo, A. Joshi, A. Mandlekar, and Y. Zhu, “RoboCasa: Large-scale simulation of everyday tasks for generalist robots,” *arXiv preprint arXiv:2406.02523*, 2024.
- [34] M. D. Zeiler and R. Fergus, “Visualizing and understanding convolutional networks,” in *European Conference on Computer Vision*. Springer, 2014, pp. 818–833.